CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

NAME:P.POORNA CHANDRA

REG NO:1924722287

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

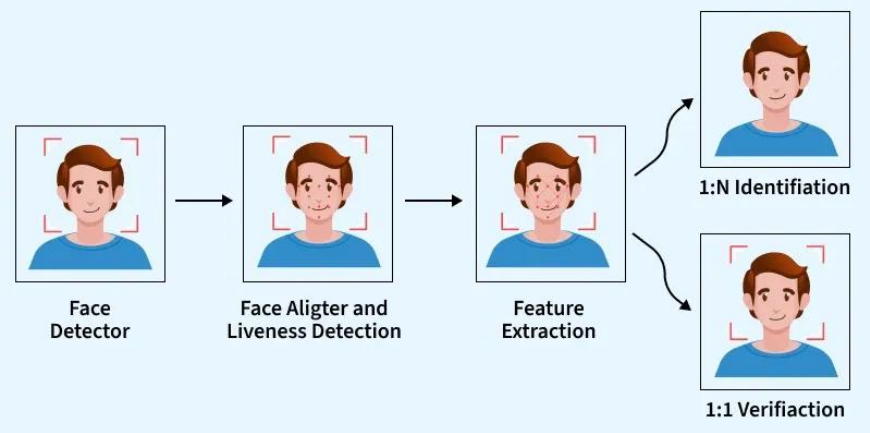
- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

1. Real-Time Face Detection and Recognition – Pseudocode

Introduction

A real-time face detection and recognition system is widely used in security applications to identify authorized individuals from live video streams. The system continuously captures frames from a camera, detects multiple faces, extracts facial features, compares them with a trained dataset, and displays the identity of recognized persons. OpenCV provides efficient algorithms such as Haar Cascade for face detection and LBPH (Local Binary Pattern Histogram) for face recognition, enabling real-time performance even on low-cost computers.



System Flow

Start

|

▼

Initialize Camera

|

▼

Capture Video Frame

|

▼

Preprocess Image

(Resize + Grayscale + Noise Removal)

|

▼

Detect Faces

|

▼

Extract Face Region

|

▼

Recognize Face

|

▼

Draw Bounding Box & Name

|

▼

Display Frame

|

▼

Repeat Until Exit Key Pressed

Detailed Pseudocode

BEGIN

Load Haar Cascade Face Detector

Load Trained Face Recognition Model (LBPH)

Load Student/Person Database

Open Camera

WHILE Camera is Running

    Capture Frame

    IF Frame is Empty

        Continue

    ENDIF

    Resize Frame

    Convert Frame to Grayscale

    Apply Histogram Equalization

    Apply Gaussian Blur

    Detect Faces using Haar Cascade

```
    FOR each Face Detected
        Extract Face ROI
        Resize Face ROI to Standard Size
        Predict Person using LBPH Recognizer
        IF Confidence < Threshold
            Retrieve Person Name
            Draw Green Rectangle
            Display Person Name
        ELSE
            Display Unknown
            Draw Red Rectangle
        ENDIF
    ENDFOR
    Display Processed Frame
    IF ESC Key Pressed
        Exit Loop
    ENDIF
ENDWHILE
Release Camera
Destroy All Windows
END
```

Explanation of Each Step

1. Camera Initialization

The webcam or CCTV camera is initialized using OpenCV's VideoCapture function. The camera continuously provides live video frames for processing.

2. Frame Acquisition

Each video frame is captured sequentially. If the frame is invalid or empty, the system skips it and proceeds to the next frame.

3. Image Preprocessing

Before detection, preprocessing improves image quality.

- Resize image to reduce computation.

- Convert RGB image to grayscale.
- Apply histogram equalization for varying lighting.
- Apply Gaussian Blur to reduce noise.

4. Face Detection

The Haar Cascade classifier scans the grayscale image to detect all visible faces simultaneously. Each detected face is represented by its bounding rectangle coordinates.

5. Feature Extraction

Each detected face is cropped and resized to a fixed dimension. LBPH extracts texture-based facial features for recognition.

6. Face Recognition

The extracted facial features are compared with the trained dataset.

- If confidence is above the predefined threshold, the person's identity is displayed.
- Otherwise, the face is labeled as "Unknown."

7. Visualization

The system draws:

- Bounding rectangle
- Person name
- Confidence score (optional)

for every detected face.

8. Efficient Loop Control

The system continuously repeats the process until the user presses the ESC key. The camera is released and all OpenCV windows are closed properly.

Optimization Techniques

- Resize frames before processing.
- Detect faces only every few frames if CPU usage is high.
- Track detected faces between frames.
- Use grayscale images instead of RGB.
- Load trained model only once during initialization.

These optimizations reduce computation and maintain real-time performance.

Advantages

- Detects multiple faces simultaneously.
- Works in real time.
- Low computational requirements.
- High recognition accuracy.
- Suitable for campus security and attendance systems.

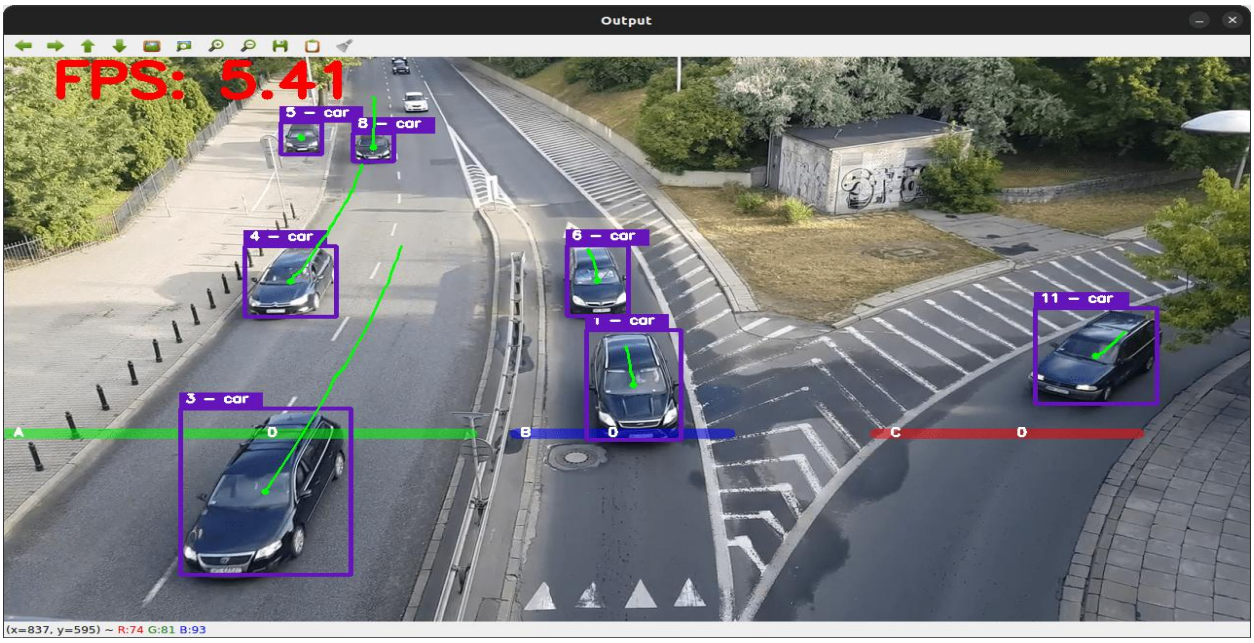
CONCLUSION

Real-time face detection and recognition helps identify people quickly and accurately, making it useful for security and surveillance systems.

2. Vehicle Detection, Tracking and Counting – Pseudocode

Introduction

An intelligent traffic monitoring system automatically detects vehicles, tracks their movement across frames, counts vehicles crossing a predefined line, and displays the results in real time. OpenCV provides background subtraction, contour detection, and tracking algorithms to perform these tasks efficiently.



System Flow

Start

Initialize Video

Capture Frame

Preprocess Frame

Detect Moving Vehicles

Track Vehicles

Check Line Crossing

Update Count

Display Results

Repeat Until Exit

Detailed Pseudocode

```
BEGIN
Open Video Stream
Initialize Background Subtractor (MOG2)
Initialize Vehicle Counter = 0
Initialize Empty Tracking List
Draw Virtual Counting Line
WHILE Video is Running
    Capture Frame
    IF Frame is Empty
        Break
    ENDIF
    Resize Frame
    Apply Gaussian Blur
    Apply Background Subtraction
    Apply Threshold
```

```
Apply Morphological Opening
Apply Morphological Closing
Find Contours
FOR each Contour
    IF Contour Area < Minimum Size
        Ignore Contour
    ELSE
        Compute Bounding Box
        Compute Vehicle Centroid
        Match Vehicle with Existing Tracking ID
        IF Match Found
            Update Vehicle Position
        ELSE
            Create New Tracking ID
        ENDIF
        Draw Bounding Box
        Display Tracking ID
        IF Vehicle Crosses Counting Line
            IF Vehicle Not Counted Before
                Vehicle Counter = Vehicle Counter + 1
                Mark Vehicle as Counted
            ENDIF
        ENDIF
    ENDIF
ENDFOR
Remove Lost Tracking IDs
Display Vehicle Count
Display Processed Frame
IF ESC Key Pressed
    Exit Loop
ENDIF
```

ENDWHILE

Release Video

Destroy All Windows

END

Explanation of Each Step

1. Video Acquisition

The system continuously captures frames from a roadside surveillance camera.

2. Preprocessing

The captured image is prepared before detection.

- Resize image
- Gaussian Blur removes camera noise
- Convert moving objects into foreground using MOG2

3. Vehicle Detection

Foreground regions are obtained using background subtraction.

Contours are extracted from the binary image.

Very small contours are ignored because they represent noise rather than vehicles.

4. Vehicle Tracking

Each detected vehicle is assigned a unique tracking ID.

The centroid position is compared with previous frames.

Existing IDs are updated while new vehicles receive new IDs.

5. Vehicle Counting

A virtual line is placed across the road.

Whenever a vehicle centroid crosses the line:

- Check whether it has already been counted.
- If not counted previously, increment the counter.

This prevents double counting.

6. Visualization

The system displays:

- Vehicle bounding box
- Tracking ID

- Vehicle count
- Counting line

All information is updated continuously.

7. Efficient Loop Termination

The program runs continuously until:

- Video ends, or
- User presses the ESC key.

The camera is released and OpenCV windows are closed.

Optimization Techniques

- Resize video frames.
- Ignore very small contours.
- Use centroid tracking instead of expensive deep trackers.
- Remove inactive tracking IDs.
- Apply morphological operations to reduce noise.
- Process only foreground regions.

These techniques improve both speed and counting accuracy.

Advantages

- Accurate vehicle counting.
- Prevents duplicate counting.
- Handles multiple vehicles simultaneously.
- Near real-time performance.
- Suitable for smart traffic monitoring systems.

CONCLUSION: Vehicle detection, tracking, and counting helps monitor vehicle movement and traffic flow efficiently, making it useful for intelligent transportation systems.